



Funded by the
European Union
NextGenerationEU



European
Digital Innovation
Hubs Network



Experiments

Helmut Simonis

Constraint Based Production Scheduling



This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only. If others modify or adapt the material, they must license the modified material under identical terms.



Acknowledgments



This publication was developed as part of the ENTIRE EDIH project, which received funding from Enterprise Ireland and the European Commission.

Part of this work is based on research conducted with the financial support of Science Foundation Ireland under Grant number 12/RC/2289-P2 at Insight the SFI Research Centre for Data Analytics at UCC, which is co-funded under the European Regional Development Fund.

Part of this work is based on research conducted within the ASSISTANT European project, under the framework program Horizon 2020, ICT-38-2020, Artificial intelligence for manufacturing, grant agreement number 101000165.

Key Points



- This section describes the scheduling tool
 - This is a *demonstrator*, not an industrial product!
- How to load/create data
 - From files
 - By instance generator
 - From benchmark problems
- How to run the solvers
 - Which solvers are supported
 - What to expect in terms of performance
- Experiments to try
 - Limited time
 - Possible "test before invest" continuation

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Some Suggested Experiments

Summary

The Scheduling Tool



- We create the tool as basis for experiments
- To test ideas and solvers
- As a teaching tool
- Slightly higher standard than usual academic prototypes
 - This is a *demosntrator*
- Not a commercial tool
 - But can use commercial solvers
 - Also open-source solvers
- Written in Java, JavaFX
- Can also be used as a back-end scheduling server
- Uses our Java application framework generator
- Sources available on request

Outline



The Scheduling Tool

Under the hood

- Google CP Sat

- IBM CPOptimizer

- MiniZinc

- Which solver is better?

Input Data

Result Output

Instance Generator

Predefined Problem Sets



- Provide both open-source and commercial solver interfaces
- Allow experimentation without having to buy commercial tools straightaway
- Gives a level playing field to compare solvers and models
- Provides out-of-the-box, generic performance

- Open-Source tool provided by Google
- Available at https://developers.google.com/optimization/cp/cp_solver
- Probably best open-source CP solver for scheduling
- This solver is packaged with scheduler

Example Problem

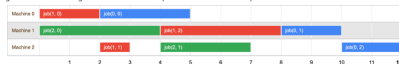
Below is a simple example of a job shop problem, in which each task is labeled by a pair of numbers (m, p) where m is the number of the machine the task must be processed on and p is the processing time of the task – the amount of time it requires. (The numbering of jobs and machines starts at 0.)

- Job 0 = $\{(0, 3), (1, 2), (2, 2)\}$
- Job 1 = $\{(0, 2), (2, 1), (1, 4)\}$
- Job 2 = $\{(1, 4), (2, 3)\}$

In the example, job 0 has three tasks. The first, $(0, 3)$, must be processed on machine 0 in 3 units of time. The second, $(1, 2)$, must be processed on machine 1 in 2 units of time, and so on. Altogether, there are eight tasks.

A solution for the problem

A solution to the job shop problem is an assignment of a start time for each task, which meets the constraints given above. The diagram below shows one possible solution for the problem:



You can check that the tasks for each job are scheduled at non-overlapping time intervals, in the order given by the problem.

The length of this solution is 12, which is the first time when all three jobs are complete. However, as you will see [below](#), this is not the optimal solution to the problem.

(from OR-Tools website)

CP Optimizer from IBM



- Commercial tool of IBM
- <https://www.ibm.com/products/ilog-cplex-optimization-cplex-cp-optimizer>
- Part of optimization suite with Cplex, OPL
- We do **not** provide this solver, we allow to interface with it
- Academic licenses available
- Well-known for capabilities for scheduling

Resources



Applications of constraint programming

Explore applications of constraint programming including production problem and scheduling use cases.

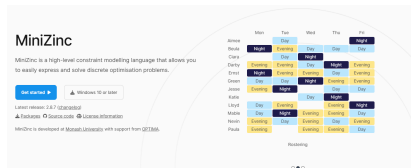
[Read the documentation](#) →

(from CPOptimizer website)

MiniZinc from Monash University



- Modelling language and backend tools from Monash University in Melbourne, Australia
- Available from <https://www.minizinc.org/>
- Widely used for teaching
- Allows different backend solver to run from same model
- Generic CP tool, not optimized for scheduling
- Requires separate installation, open-source



(from MiniZinc Website)

Which Solver is Better?



- We present results on a few benchmark types
- Fair comparison between solvers
 - Same hardware, Windows 11 laptop
 - CPU i7-10875H @ 2.3GHz, 64GB, four cores
 - Same timeout (600 s)
- Not a fair comparison to state-of-the-art
 - Uses out-of-the-box model
 - Significant improvements possible
 - More specific models
 - Parameter tuning
 - Unlimited runtime

Taillard Job-Shop Benchmarks



Group	Nr	All Instances				Optimal Only		Non Optimal Only			
		Optimal (% of All Instances)				Time (% of VB)		Cost (% of VB)		Bound (% of VB)	
		Both	CPO	CPSat	None	CPO	CPSat	CPO	CPSat	CPO	CPSat
15/15	10	90.00	0.00	0.00	10.00	105.19	141.18	100.00	100.00	97.17	100.00
20/15	10	20.00	0.00	0.00	80.00	267.27	263.20	100.99	100.05	98.50	99.93
20/20	10	0.00	0.00	0.00	100.00	n/a	n/a	100.74	100.06	97.96	100.00
30/15	10	10.00	0.00	10.00	80.00	174.32	100.00	100.18	100.49	99.87	100.00
30/20	10	0.00	0.00	0.00	100.00	n/a	n/a	100.30	101.30	99.40	100.00
50/15	10	100.00	0.00	0.00	0.00	100.00	685.09	n/a	n/a	n/a	n/a
50/20	10	10.00	60.00	0.00	30.00	100.00	381.38	100.00	101.60	100.00	100.00
100/20	10	10.00	90.00	0.00	0.00	100.00	416.13	100.00	101.73	100.00	66.81

- Significant number of problems solved to optimality in 600s
- In terms of quality, solvers are quite similar
- CPO wins in terms of solution times for larger instances

Results for Hybrid Flexible Flow-Shop



Group	Nr	All Instances				Optimal Only		Non Optimal Only			
		Optimal (% of All Instances)				Time (% of VB)		Cost (% of VB)		Bound (% of VB)	
		Both	CPO	CPSat	None	CPO	CPSat	CPO	CPSat	CPO	CPSat
20	25	76.00	0.00	20.00	4.00	100.00	580.71	100.00	100.00	96.52	100.00
25	25	80.00	0.00	8.00	12.00	101.65	238.02	100.00	100.37	97.67	100.00
30	25	60.00	0.00	4.00	36.00	100.35	264.69	100.18	101.05	100.00	100.00
40	25	4.00	16.00	0.00	80.00	100.00	2554.03	100.00	104.68	100.00	100.00
50	25	0.00	4.00	0.00	96.00	n/a	n/a	100.00	107.87	100.00	100.00
100	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	120.43	100.00	100.00
200	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	188.60	100.00	100.00
300	24	0.00	0.00	0.00	100.00	n/a	n/a	100.00	263.22	100.00	100.00
400	25	0.00	0.00	0.00	100.00	n/a	n/a	100.00	246.34	100.00	100.00

- Only smaller/medium instances solved to optimality
- For those problems, both solvers perform well
- CPO significantly better on large instances

General Recommendations



- If you already have access to CPO, use it!
- For new problem types, do an evaluation with CPSat first
- Out of the box, CPO performs more consistently
- May be easier to extend CPSat with your own research
- Use multiple cores and memory to your advantage

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

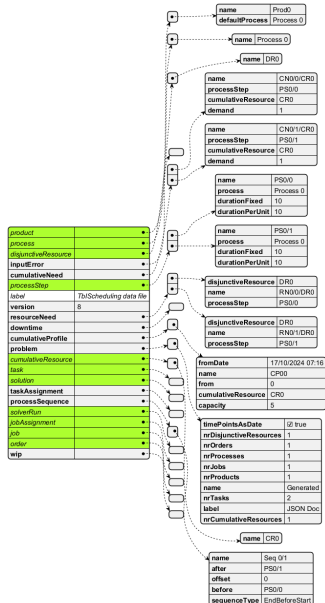
Some Suggested Experiments

Summary



- We have defined a specific JSON data format to describe scheduling problems
- This is different from the native/XML data format of the application (do not use)
- Load with menu `File - Load DataFile...`
- Save with menu `File - Save DataFile...`
- The format is described in a document

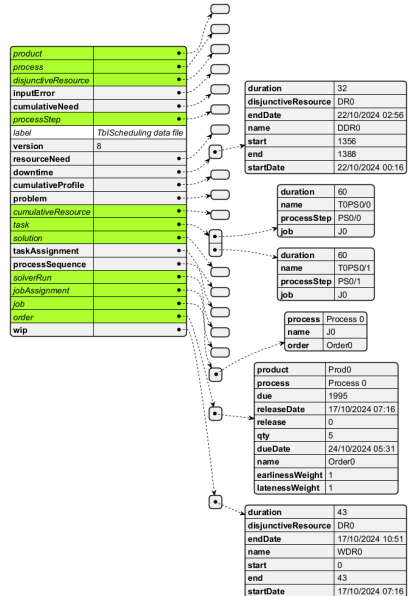
- Description of
 - Product
 - Process
 - DisjunctiveResource
 - CumulativeNeed
 - ProcessStep
 - ResourceNeed
 - CumulativeProfile
 - Problem
 - CumulativeResource
 - ProcessSequence



Schedule Input Data



- Description of
 - Downtime
 - Task (x2)
 - Job
 - Order
 - WiP



Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Some Suggested Experiments

Summary



- We use the same JSON format to describe the results of the schedule
- Added field types for SolverRun, Solution, assigned Jobs and Tasks

Sample Results



- Description of
 - Solution
 - SolverRun
 - Job Assignment
 - Task Assignment

product	
process	
dispatchedResource	
inputTime	
outputTime	
resourceType	
start	7/15/2024 15:31
version	0
resourceUsed	
downTime	
outputResource	
output	
outputResource	
task	
taskName	
taskAssignment	
resourceSequence	
dispatched	
jobAssignment	
job	
unit	
unit	

duration	183
totalDuration	1832
startTime	17/10/2024 20:51
totalWaitTime	0
priority	1
percentage	100
duration	129
maxLatency	0
maxWaitTime	0
maxLat	0
job	0
jobRun	Run1
end	183
weightOfDuration	1832
totalLatency	0
maxWaitTime	0
total	183
resourceType	Cyclist
start	43
totalWaitTime	0
percentage	0
weightOfLatency	1832
maxLatency	0
name	Set1
dispatchedTime	183
startTime	17/10/2024 10:51

duration	80
waitTime	0
task	TOP00/1
dispatchedResource	DR0
endTime	17/10/2024 15:31
name	TA0
start	43
end	103
jobAssignment	JAO
waitTime	0
startTime	17/10/2024 15:31

duration	80
waitTime	0
task	TOP00/1
dispatchedResource	DR0
endTime	17/10/2024 20:51
name	TA1
start	103
end	183
jobAssignment	JAO
waitTime	0
startTime	17/10/2024 15:31

inThrottle	2
weightOfLatency	1
end	42
outputResource	None
weightOfLatency	1
resourceType	Cyclist
description	
label	
resourceType	CPO
resourceType	W100
resourceType	Multiplan
resource	20
weightOfLatency	1
resourceCumulative	20
name	Run1
weightOfLatency	1
time	8.142
resourceReport	Cyclist
resourceDownTime	W100
resourceDownTime	Cyclist
resourceDownTime	W100
resourceDownTime	W100
resourceDownTime	W100

duration	129
solution	Set1
time	0
endTime	17/10/2024 20:51
name	JAO
start	43
end	183
job	20
unit	1832
startTime	17/10/2024 10:51

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Some Suggested Experiments

Summary



- Application allows to generate different types of test problems
- Different types of resource models
- Different numbers of orders, resources, WiP, downtime
- Useful to generate more life-like examples combining different constraint types

Instance Generator Dialog



- Resource Model
 - Select a resource model defining the overall structure of problem
- Nr Disjunctive Resources
 - Describe how many disjunctive resources are generated
- Resource Probability
 - The probability that a resource is compatible with a task
 - Only for some resource models

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. The 'Label' field is empty. The 'StartDate' is set to '01/10/2024' and the 'Start Time' is '0 : 00'. The 'Resource Model' is set to 'HybridFlowShop'. The 'Nr Disjunctive Resources' slider is set to 7. The 'Resource Probability' slider is set to 0.3. The 'Cumulative Resource' section is expanded, showing 'Nr Cumulative Resources' set to 4, 'Cumul Demand Range' set to 1, 'Profile Pieces' set to 3, and 'Cumul Capacity Range' set to 6. The 'Orders', 'WIP', 'Downtime', and 'Other Parameters' sections are collapsed. 'Run' and 'Cancel' buttons are at the bottom right.

Resource Models



- Flow-Shop
 - Multiple stages, all jobs use machines in same order
- Job-Shop
 - Multiple stages, jobs use machines in different order
- Open-Shop
 - Multiple stages, no predefined order of machines
- Hybrid Flow-Shop (default)
- Hybrid Job-Shop
- Hybrid Open-Shop
 - Like x-shop, but with multiple machines per stage
- Random
 - Multiple stages, each stage using a random subset of machines
- All
 - Multiple stages, each stage allowing all machines

Instance Generator - Products



- Nr Products
 - Number of products to be generated
 - Products may be reused by multiple orders
- Stages Range
 - Range slider, sets lower and upper bound on number of stages

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. The 'Label' field is empty. The 'StartDate' is set to '01/10/2024' and the 'Start Time' is '0 : 00'. The 'Resource Model' is set to 'HybridFlowShop'. The 'Nr Disjunctive Resources' slider is set to 7. The 'Resource Probability' slider is set to 0.3. The 'Products' section is expanded, showing 'Nr Products' set to 10 and 'Stages Range' set to 4. Other sections like 'Duration', 'Cumulative Resource', 'Orders', 'WIP', 'Downtime', and 'Other Parameters' are collapsed. 'Run' and 'Cancel' buttons are at the bottom right.

Data Generator Parameters

Label:

StartDate: Start Time:

Resource Model:

Nr Disjunctive Resources:

Resource Probability:

Products

Nr Products:

Stages Range:

► Duration

► Cumulative Resource

► Orders

► WIP

► Downtime

► Other Parameters

Run Cancel

Instance Generator - Duration



- Duration Model
 - Different ways to link duration of processSteps
- Duration Range
 - Range slider to set lower and upper bounds on perUnit duration
- Duration Fixed Factor
 - How fixed and perUnit duration values are linked

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. At the top, there is a 'Label' field, a 'StartDate' field set to '01/10/2024', and a 'Start Time' field set to '0 : 00'. Below these is a 'Resource Model' dropdown menu set to 'HybridFlowShop'. There are two sliders: 'Nr Disjunctive Resources' with a range from 1 to 20 and a value of 7, and 'Resource Probability' with a range from 0 to 1 and a value of 0.3. A sidebar on the left lists categories: 'Products' (expanded), 'Duration', 'Cumulative Resource', 'Orders', 'WIP', 'Downtime', and 'Other Parameters'. The 'Duration' section is active, showing a 'Duration Model' dropdown set to 'Random', a 'Duration Range' slider from 1 to 50 with a value of 16, and a 'Duration Fixed Factor' slider from 0 to 5 with a value of 1. At the bottom right are 'Run' and 'Cancel' buttons.

Instance Generator - Cumulative



- Nr Cumulative Resources
 - Number of cumulative resources generated
- Cumul Demand Range
 - Range slider to select lower and upper bound on cumulativeResource-Need demands
- Profile Pieces
 - Number of segments of CumulativeProfile generated for each resource
- Cumul Capacity Range
 - Range slider to select lower and upper bounds on cumulative profile

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. The 'Label' field is empty. 'StartDate' is set to '01/10/2024' and 'Start Time' is '0 : 00'. 'Resource Model' is set to 'HybridFlowShop'. 'Nr Disjunctive Resources' is a slider from 1 to 20, currently at 7. 'Resource Probability' is a slider from 0 to 1, currently at 0.3. The 'Cumulative Resource' section is expanded, showing 'Nr Cumulative Resources' (slider 1-5, at 3), 'Cumul Demand Range' (slider 1-10, at 3), 'Profile Pieces' (slider 1-5, at 3), and 'Cumul Capacity Range' (slider 1-30, from 6 to 11). Other sections like 'Products', 'Duration', 'Orders', 'WiP', 'Downtime', and 'Other Parameters' are collapsed. 'Run' and 'Cancel' buttons are at the bottom right.

Instance Generator - Orders



- Nr Orders
 - Number of orders generated, each order is assigned a random product/process
- Qty Range
 - Range slider to select lower and upper bounds on quantity for each order

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. The 'Label' field is empty. The 'StartDate' is set to '01/10/2024' and the 'Start Time' is '0 : 00'. The 'Resource Model' is set to 'HybridFlowShop'. The 'Nr Disjunctive Resources' slider is set to 7. The 'Resource Probability' slider is set to 0.3. The 'Orders' section is expanded, showing 'Nr Orders' set to 50 and a 'Qty Range' slider set from 1 to 11. Other sections like 'Products', 'Duration', 'Cumulative Resource', 'WIP', 'Downtime', and 'Other Parameters' are collapsed. 'Run' and 'Cancel' buttons are at the bottom right.

Data Generator Parameters

Label:

StartDate: Start Time:

Resource Model:

Nr Disjunctive Resources:

Resource Probability:

► Products

► Duration

► Cumulative Resource

▼ Orders

Nr Orders:

Qty Range:

► WIP

► Downtime

► Other Parameters

Run Cancel

Instance Generator - WiP (Work in Progress)



- WiP Probability
 - Probability of generating a WiP for a disjunctive resource
- WiP Range
 - Range slider to set lower and upper bound on WiP duration

A screenshot of the 'Data Generator Parameters' dialog box. It contains several input fields and sliders. At the top, there is a 'Label' field. Below it, 'StartDate' is set to '01/10/2024' and 'Start Time' is '0 : 00'. The 'Resource Model' is set to 'HybridFlowShop'. There are two sliders: 'Nr Disjunctive Resources' with a value around 7, and 'Resource Probability' with a value around 0.3. A list of categories is shown: 'Products', 'Duration', 'Cumulative Resource', 'Orders', 'WiP' (selected), 'Downtime', and 'Other Parameters'. Under the 'WiP' category, there is a 'WiP Probability' slider set to 1 and a 'WiP Range' slider with a blue bar between 21 and 51. At the bottom right are 'Run' and 'Cancel' buttons.

Instance Generator - Downtime



- Downtime Probability
 - Probability of generating a downtime for a disjunctive resource
- Downtime Range
 - Range slider to select lower and upper bounds on downtime duration

A screenshot of the 'Data Generator Parameters' dialog box. It contains fields for 'Label', 'StartDate' (01/10/2024), and 'Start Time' (0 : 00). The 'Resource Model' is set to 'HybridFlowShop'. There are two sliders: 'Nr Disjunctive Resources' (range 1 to 20, value at 7) and 'Resource Probability' (range 0 to 1, value at 0.3). A list of categories includes Products, Duration, Cumulative Resource, Orders, WIP, and Downtime (selected). Under 'Downtime', there is a 'Downtime Probability' slider (range 0 to 1, value at 0.5) and a 'Downtime Range' slider (range 1 to 100, values at 51 and 71). An 'Other Parameters' section is also visible. 'Run' and 'Cancel' buttons are at the bottom right.

Instance Generator - Other Parameters



- Earliest Due
 - Smallest allowed value for a due date
- Horizon Days
 - What planning horizon to consider (in days)
- Time Resolution
 - In minutes, links internal and external time presentation
- Random Seed
 - Random seed to make reproducible random choices

A screenshot of the 'Data Generator Parameters' dialog box. It contains fields for 'Label', 'StartDate' (01/10/2024), 'Start Time' (0 : 00), 'Resource Model' (HybridFlowShop), 'Nr Disjunctive Resources' (slider from 1 to 20, set at 7), and 'Resource Probability' (slider from 0 to 1, set at 0.3). A tree view on the left shows categories: Products, Duration, Cumulative Resource, Orders, WIP, Downtime, and Other Parameters. The 'Other Parameters' section is expanded, showing 'Earliest Due' (20), 'Horizon Days' (20), 'Time Resolution' (5), and 'Random Seed' (42). 'Run' and 'Cancel' buttons are at the bottom right.

Data Generator Parameters

Label:

StartDate: Start Time:

Resource Model:

Nr Disjunctive Resources:

Resource Probability:

Products

Duration

Cumulative Resource

Orders

WIP

Downtime

Other Parameters

Earliest Due:

Horizon Days:

Time Resolution:

Random Seed:

Run Cancel

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Taillard

SALBP

Test Scheduling



- Three datasets of different sizes
 - Job-shop
 - Flow-shop
 - Open-shop
- Load with menu `File - Load DataFile...` -
Taillard -
- Larger instances need more solver time to reach good solutions (600 s)

Simple Assembly Line Balancing Problem (SALBP)



- Will be discussed on more details as case study
- Design an assembly line setup by solving a scheduling problem
- Balance a set of operations across a number of stations of an assembly line
- Precedence graph is not a chain, can be very complex
- Specialized problem normally solved with specialized tools
- Load with menu `File - Load SALBP Problem...`

Test Scheduling Benchmark set from ABB



- Will be discussed in more details as case study
- Schedule a set of tests on a number of machines, minimizing total duration
- Single stage tests, possibly large number of resources
- Closely related to bin-packing
- Load with menu `File - Load Test Scheduling Problem...`

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Some Suggested Experiments

Summary

Experiment 1



- Start the application
 - Our running example will be automatically generated
- Look at the process diagram `Window-Product-Process Diagram`
- Run the solver `Scenario - Run ScheduleJobs Solver`
- Observe the results in Gantt Chart
- Customize display
- Look at Cumulative Resource Chart
`Window-Solution-Cumulative Resource Chart`

Experiment 2



- Re-run solver disabling cumulative constraint
- Observe result in Gantt chart
- See impact on Cumulative Resource chart
- Switch between solutions in charts

Experiment 3



- Check Gantt chart display for delayed tasks, enabling lateness display
- Re-run solver, enforcing due-date constraints
- What impact does this have on objective

Experiment 4



- Change objective to on-time delivery
- Results are very different, why?
- More explanations on this tomorrow

Experiment 5



- Load one of the other example types
- For example, Taillard Job-shop 15x15
- Understand process diagram
- Run solver
- Look at intermediate solutions found

Outline



The Scheduling Tool

Under the hood

Input Data

Result Output

Instance Generator

Predefined Problem Sets

Some Suggested Experiments

Summary

Summary



- We presented an overview of our generic scheduling tool
- Discussed available solvers, both commercial and open-source
- Described the JSON data format for input and output
- Gave an overview of the instance generator provided
- Shows example problems included with tool
- Suggested some experiments to run